Day 11 Student Notes: Claude for Docker

> **Today in one line:** pack an app with Docker, fix a bloated and risky Dockerfile with Claude, and **check the result with real commands**.

Docker in plain words

Word	Meaning
Image	A sealed lunchbox: the app and everything it needs (read only)
Container	The opened, running lunchbox: a running copy of an image
Dockerfile	The packing list that builds an image
Layer	One step of the build. Docker reuses unchanged layers (cache)

```
Dockerfile  --docker build-->  image  --docker run-->  container
```

Dockerfile lines

Line	What it means
FROM	Start from this base image
WORKDIR	The folder inside the container to work in
COPY	Copy files from your folder into the image
RUN	Run a command while building
ENV	Set a setting inside the image
EXPOSE	Note which port the app uses
USER	Which user runs the app
CMD	The command that starts the app

A small Dockerfile, line by line

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

1. Start from a small Python image.
2. Work inside the /app folder.
3. Copy the list of packages.
4. Install them.
5. Copy the app code.
6. The app listens on port 5000.
7. Start the app.

Packages come before the code, so a code change reuses the slow install step (layer caching).

Commands

```
docker build -t name .      make an image
docker images               list images and their size
docker run -d -p 5000:5000 --name web name
docker ps                   running containers (ps -a: all)
docker logs web             see what it printed
docker exec web id          run a command inside
docker rm -f web            stop and remove
```

-p 5000:5000 means host port : container port. The left number is the port on your laptop, the right number is the port inside the container. Only look: images, ps, logs. rm -f stops and removes, so read before you run.

What was wrong with the bad Dockerfile

Problem	Why it matters
ubuntu:latest and many tools	Big image, more to attack, a different base each build
COPY . takes .env	Secrets end up inside the image
ENV SECRET_KEY=...	The secret is visible to anyone who has the image
No USER line	Runs as root, the most powerful user
EXPOSE 22	A port nothing uses

The fix: multi-stage, non-root

```
# Stage 1: install the Python packages
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2: the small image we actually run
FROM python:3.12-slim
RUN useradd --create-home --uid 10001 appuser
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:5000/health')"
CMD ["python", "app.py"]
```

- **Stage 1** (the kitchen) installs the packages. The heavy tools stay there.
- **Stage 2** (the clean plate) starts fresh and copies only what the app needs.
- **USER appuser** means not root.
- **.dockerignore** keeps .env, .git and junk out of the image.

For a small Python app, most of the size saving comes from the slim base image. Multi-stage saves much more for apps that compile code, such as Java, Go or Node builds.

The steps and prompts (also in Day11_Prompts_To_Paste.txt)

Step 1: explain the bad Dockerfile

```
Explain this Dockerfile line by line in simple words.
Then list everything wrong or risky about it, most serious first.
Do not change any file.
```

Step 2: build the bloated image and see the problems

```
docker build -t shop-status:bloated .
docker images
docker run -d --name bloated -p 5000:5000 shop-status:bloated
curl localhost:5000
docker exec bloated id
docker exec bloated ls -a /app
docker rm -f bloated
```

You should see: a big size in docker images; **uid=0(root)**; and **.env** inside the image.

Step 3: Claude fixes it

```
Rename my Dockerfile to Dockerfile.bloated. Then write a new Dockerfile for the
same app: small, multi-stage, and running as a non-root user.
Also create a .dockerignore so .env and other junk stay out of the image.
Explain each change in one line.
```

Step 4: build the lean image and compare

```
docker build -t shop-status:lean .
docker images
docker run -d --name lean -p 5000:5000 shop-status:lean
curl localhost:5000
docker exec lean id
docker exec lean ls -a /app
docker rm -f lean
```

Compare: much smaller size, uid=10001(appuser) not root, no .env, and curl still works. Then:

```
Check the new Dockerfile against this list and say PASS or FAIL for each:
non-root user, no secrets, small base image, pinned version (no :latest),
.dockerignore, healthcheck.
Do not change anything.
```

Step 5: the container will not start

```
docker run -d --name oops -e PORT=abc shop-status:lean
docker ps -a
docker logs oops
docker rm -f oops
```

Wait three seconds after the first command. docker ps -a shows **Exited (1)**, and docker logs shows the error. We gave the app PORT=abc, and a port must be a number. Then ask Claude:

```
My container stopped right after I started it.
Here are the logs:
[paste the docker logs output here]
What is wrong and how do I fix it? Do not change any file yet.
```

Debug loop for containers

1. **docker ps -a**: is it running, or did it exit?
2. **docker logs name**: what does the error say?
3. Ask Claude with the logs. Do not paste real secrets.
4. Fix, run again, check.

Key words

Word	Meaning
Multi-stage build	Build in one stage, copy only the result into a small final image
Non-root	The app runs as a normal user, not the most powerful one
.dockerignore	A list of files Docker must not copy into the image
Pinned version	An exact version like python:3.12-slim instead of latest
Port mapping	-p host:container connects a laptop port to a container port
Healthcheck	A test Docker runs to see if the app is healthy

Check yourself

1. What is the difference between an image and a container?
2. Name three problems in the bad Dockerfile.
3. Why do we copy requirements.txt before the code?
4. What does -p 5000:5000 mean?
5. A container stopped right after starting. What are the first two commands?

Answers

1. An image is the packed app. A container is a running copy of it.
2. Any three of: :latest, many tools, secrets in the image, root user, port 22, copying .env.
3. So Docker reuses the slow install layer when only the code changes.
4. Host port 5000 connects to container port 5000.
5. docker ps -a, then docker logs.

Take-home task (30 minutes)

1. Ask Claude to write a docker-compose.yml that builds the folder and runs the app on port 5000 (prompt below).
2. Run it with **docker compose up**, open localhost:5000, then stop it with **docker compose down**.
3. Ask Claude to explain each line of the file in simple words.

```
Write a docker-compose.yml that builds this folder and runs the app on port 5000.
Explain each line in simple words.
```

Fake data only. Never put real passwords in a Dockerfile or .env that you share.

> **Next class, Day 12:** Claude for Kubernetes: running containers at scale.